

Data Structures and Algorithms

Instructor: Dr. Hao Qin (秦浩)

E-mail: hao.qin@scupi.cn

My Office: Room 514, SCUPI (North)

Sorting!

Sorting is the process of ordering a bunch of **items** based on one or more **rules**, subject to one or more **constraints**...

Items - what are we sorting,
and how many are there?

- Strings, numbers, student records, C++ objects (e.g., Circles, Robots)
- Thousands, millions or trillions?

Rules - how do we order them?

- Ascending  vs. Descending  order
- Based on Circle radius? Student GPA?
- Based on multiple criteria, e.g.:
by last name, then first name

Constraints?

- Are the items in RAM or on disk?
- Is the data in an array or a linked list?

2 Rules of Sorting

Rule #1:

Don't choose a sorting algorithm until you understand the requirements of your problem.

Rule #2:

Always choose the simplest sorting algorithm possible that meets your requirements.

Selection Sort - Better or Worse?

Are there any kinds of input data where Selection Sort is either **more** or **less efficient**?

For example, what if all of the books are **mostly in order** before our sort starts?

```
void selectSort(shelf of N books)
{
  for i = 1 to N
  {
    find the smallest book
      between slots i and N

    swap this smallest book
      with book i;
  }
}
```



No! Selection sort takes just as many steps either way!

The Selection Sort

And here's the C++ source code to sort a bunch of numbers...

```
void selectionSort(int A[], int n)
{
    for (int i = 0; i < n; i++)
    {
        int minIndex = i;
        for (int j = i+1; j < n; j++)
        {
            if (A[j] < A[minIndex])
                minIndex = j;
        }
        swap(A[i], A[minIndex]);
    }
}
```

} - For each of the n array elements...

} - Locate the smallest item in the array between the i^{th} slot and slot $n-1$.

} - Swap the smallest item found with slot $A[i]$.

The Quicksort Algorithm

1. If the array contains only 0 or 1 element, **return**.
2. Select an arbitrary element **P** from the array (typically the **first element** in the array).
3. Move all elements that are **less than or equal** to **P** to the **left of the array** and all elements **greater than P** to the **right** (this is called **partitioning**).
4. Recursively repeat this process on the left sub-array and then the right sub-array.

Divide

Conquer

13	1	21	30	69	40	77
----	---	----	----	----	----	----

D&C Sorts: Quicksort

And here's an actual Quicksort C++ function:

0 7

```
void QuickSort(int Array[],int First,int Last)
{
    if (Last - First >= 1 )
    {
        int PivotIndex;

        3
        PivotIndex = Partition(Array,First,Last) ;
        QuickSort (Array,First,PivotIndex-1); // left
        QuickSort (Array,PivotIndex+1,Last);  // right
    }
}
```

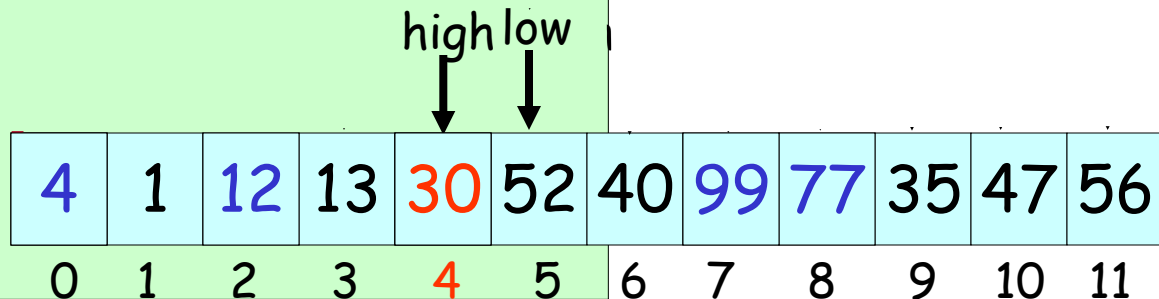
13	1	21	30	69	40	77	46
0	1	2	3	4	5	6	7

The QS Partition Function

The **Partition** function uses the first item as the pivot value and moves **less-than-or-equal items** to the **left** and **larger ones** to the **right**.

```
int Partition(int a[], int low, int high)
{
    int pi = low;
    int pivot = a[low];
    do
    {
        while ( low <= high && a[low] <= pivot )
            low++;
        while ( a[high] > pivot )
            high--;
        if ( low < high )
            swap(a[low], a[high]);
    }
    while ( low < high );
    swap(a[pi], a[high]);
    pi = high;
    return(pi);
}
```

And finally, return the pivot's index in the array (**4**) to the QuickSort function.



Big-oh of Quicksort

We first **partition** the array, at a cost of n steps.

Then we repeat the process for each half...

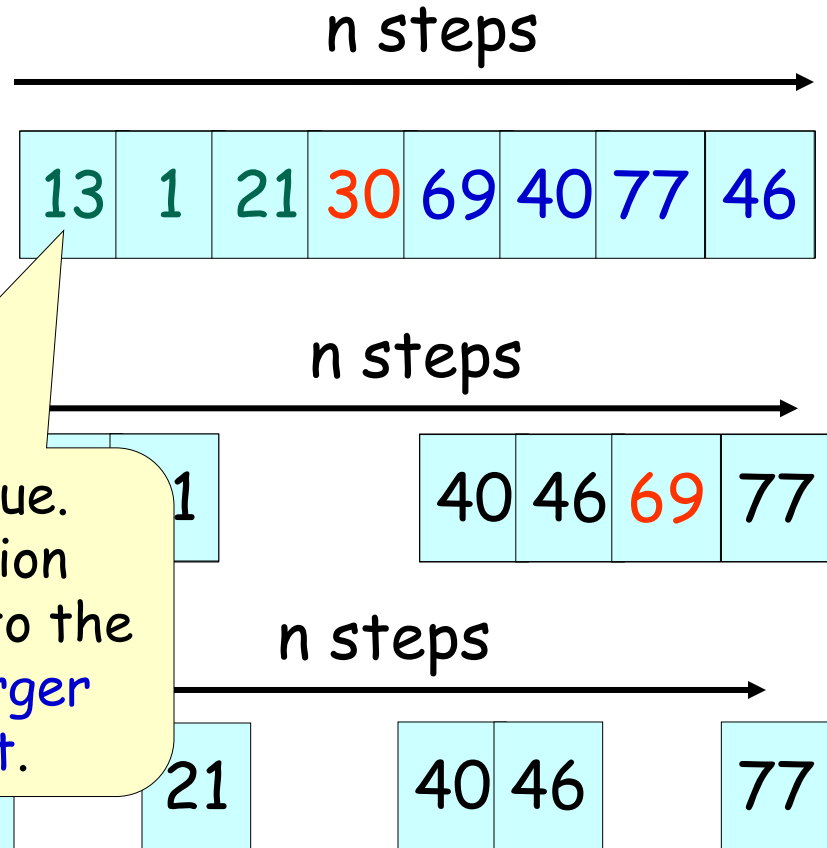
We **partition** **halves**, each at a total cost of $n/2$ steps.

Then we repeat the process for each half...

We **partition** each of the 4 **halves**, each taking $n/4$ steps, at a total cost of n steps.

$\log_2(n)$
levels

This is our **pivot** value. Our **partition** function moves **smaller values** to the **left** of the array, **larger values** to the **right**.



So at each level, we do n operations, and we have $\log_2(n)$ levels, so we get: $n \log_2(n)$.

Mergesort

The Mergesort is another extremely efficient sort - yet it's pretty easy to understand.



But before we learn the **Mergesort**, we need to learn another algorithm called "**merge**".

Mergesort

The basic **merge** algorithm takes **two-presorted arrays** as inputs and **outputs a combined, third sorted array**.



1. Initialize counter variables $i1$, $i2$ to zero
2. While there are more items to copy...
 - If $A1[i1]$ is less than $A2[i2]$
 - Copy $A1[i1]$ to output array B and $i1++$
 - Else
 - Copy $A2[i2]$ to output array B and $i2++$
3. If either array runs out, copy the entire contents of the other array over

Merge Algorithm in C++

```

void merge(int data[], int n1, int n2,
           int temp[])
{
    int i1=0, i2=0, k=0;
    → int *A1 = data, *A2 = data + n1;
    → while (i1 < n1 || i2 < n2)
    {
        if (i1 == n1)
            temp[k++] = A2[i2++];
        else if (i2 == n2)
            temp[k++] = A1[i1++];
        else if (data[i1] <= A2[i2])
            temp[k++] = A1[i1++];
        else
            temp[k++] = A2[i2++];
    }
    → for (int i=0; i<n1+n2; i++)
        data[i] = temp[i];
}

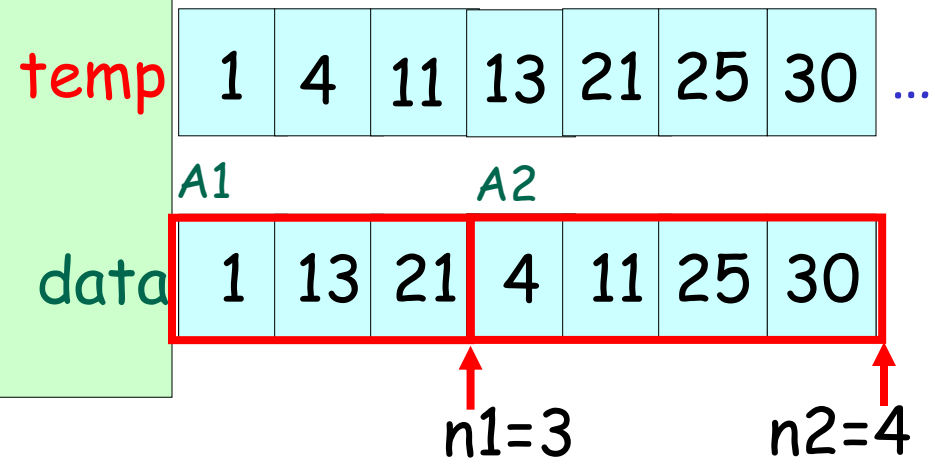
```

Here's the C++ version of our **merge** function!

You pass in an input array called **data** and the sizes of the two parts of it to merge: **n1** and **n2**

The last parameter, **temp**, is a temporary array of size $n1+n2$ that holds the merged results as we loop.

Finally, we copy our merged results back to the **data** array.



Mergesort

OK - so what's the full mergesort algorithm:

Mergesort function :

1. If array has one element, then return (it's sorted).
2. Split up the array into two equal sections
3. Recursively call Mergesort function on the left half
4. Recursively call Mergesort function on the right half
5. Merge the two halves using our **merge** function